GENERAL PROJECT QUESTIONS – Answers

What is MessFinder?

MessFinder is a web application that helps users, especially students and job seekers, to find and book PGs or rental rooms. It also allows PG owners to list their properties, making it a platform that connects seekers and providers.

Why did you build this project?

We built this project to solve the common problem of finding rental accommodation in unfamiliar areas. Most people rely on posters, agents, or social media, which are often unorganized or unsafe. MessFinder aims to simplify and secure this process.

What problem does your app solve?

It solves the difficulty of discovering verified, available PGs quickly. It reduces the dependency on unreliable sources and organizes room data in one place with direct contact options.

Who are your target users?

Our primary users are college students, new city residents, and working professionals who move frequently and need affordable accommodation. PG owners and landlords are also target users on the provider side.

What makes your project different from others?

Unlike general platforms, MessFinder is focused on PGs and student-friendly rentals. It offers role-based access, secure messaging, and an easy-to-use design. Plus, it's built with real-time data using Firebase, making it fast and reliable.

What are the core goals of your project?

- Connect room seekers with PG owners
- Provide a secure, easy-to-use platform
- Promote digital access to local PG businesses
- Eliminate scam listings by using verified submissions and chat

---

What is the real-life application of your project?

In real life, students often relocate for education and jobs. Our app helps them find PGs in advance, communicate with owners directly, and make informed decisions — saving time, effort, and reducing stress.

Is this idea inspired by any existing apps?

Yes, it's partially inspired by apps like 99acres, MagicBricks, or NoBroker — but those focus on real estate in general. MessFinder is specific to students and PGs, with simpler navigation and less commercial clutter.

How does your app help society?

It helps students and newcomers find safe, affordable housing. It supports small PG owners by giving them an online presence. It promotes digital literacy in local communities and streamlines the rental system.

What challenges did you face while building it?

- Understanding role-based data access
- Structuring Firestore collections and rules
- Implementing real-time chat securely
- Making the app responsive and accessible
- Testing forms and preventing invalid submissions

How can this project be improved in future?

We plan to add:

- Maps for nearby PGs
- Payment integration for booking

- Reviews and rating system
- Admin moderation panel
- Push notifications and mobile app version

TECHNOLOGY STACK (What / Why / How) – Answers

What tech stack did you use?

We used a **full frontend + serverless backend** stack:

- **Frontend**: React.js, Tailwind CSS, React Router, Vite
- **Backend (serverless)**: Firebase (Firestore, Auth, Storage, Hosting)
- **Deployment**: Vercel (for frontend), Firebase Hosting (optional)

This stack is fast, modern, easy to manage, and perfect for a student project.

Why did you choose React?

React is:

- Component-based, making code reusable and organized
- Supported by a large community
- Easy to combine with Firebase for dynamic UI
- Ideal for building SPAs (Single Page Applications)

We used it to build each page and feature in a modular and maintainable way.

What is Firebase and why did you use it?

Firebase is Google's serverless platform. We used it because:

- No backend server needed
- It offers secure Authentication, Realtime Database, Storage, and Hosting
- Scales automatically
- Very beginner-friendly and well-documented

It simplified complex backend tasks for our small team.

What is Tailwind CSS and how is it different from Bootstrap?

Tailwind is a utility-first CSS framework.

- Tailwind uses classes like `p-4`, `bg-blue-500`, making styling fast and flexible.
- Bootstrap uses pre-built components with limited customization.
  Tailwind gave us full design control and cleaner custom layouts for responsiveness.

What is Vercel and what does it do?

Vercel is a cloud platform to deploy frontend frameworks like React.

- It auto-deploys apps directly from GitHub
- It gives us a free HTTPS URL
- It handles caching, CDN, and builds

We used it to host and continuously deploy our frontend.

Why did you choose Vite instead of CRA?

Vite is faster than CRA (Create React App):

- Instant server startup
- Hot module replacement for quick reloads
- Smaller, optimized production builds
  CRA is older and slower. Vite makes the dev experience smoother.

What are the advantages and disadvantages of Firebase?

Advantages:

- No server maintenance
- Real-time sync
- Free tier is powerful
- Built-in Auth and Storage

Disadvantages:

- Limited query capabilities (especially for complex filters)
- Pricing increases with scale
- Vendor lock-in

But for student projects and MVPs, it's a great choice.

What does NoSQL mean in Firestore?

Firestore is a **NoSQL database**, meaning:

- Data is stored in documents inside collections (not tables/rows)
- Flexible schema — no need to define strict columns
- Great for hierarchical and real-time data (like PG listings, chats)

What is the role of Firebase Storage?

We use **Firebase Storage** to:

- Upload and store room images
- Get secure image URLs
- Control file access with storage rules

It's scalable and secure, perfect for our needs.

What is React Router used for?

React Router manages **page navigation** inside our SPA.
It lets us define paths like:

- `/` → Home
- `/details/:id` → PG Details
- `/dashboard` → Owner panel

  Without reloading the page — it's fast and smooth.

What are the benefits of using a serverless architecture?

Serverless (like Firebase) means:

- No server to configure or manage
- Scales automatically
- Pay-per-use pricing
- Easier deployment and security management

  It helped us focus more on building features instead of backend setup.

What is SPA (Single Page Application)?

SPA is an app that loads a single HTML page and dynamically updates it without full reloads.
It uses JavaScript to show different views (routes) smoothly.
React + React Router builds SPAs by default.
MessFinder is a SPA — fast and user-friendly.

FRONTEND-RELATED QUESTIONS – Answers

How did you manage routing in your app?

We used **React Router** to define and manage routes.
Each URL path (like `/`, `/details/:id`, `/dashboard`) maps to a React component.
React Router allows us to navigate between pages without reloading the entire site.

What is a React component?

A **React component** is a reusable building block of UI.
There are two types:

- **Functional components** (most common, using hooks)
- **Class components** (older, less used now)

Each page, card, form, or button group in our app is a separate component.

What is `useState` and `useEffect`?

- `useState` is a React hook that lets us create and update local state (e.g., form inputs, UI toggles).
- `useEffect` is used to run side effects — like fetching data when the component loads, or setting up listeners.

We use both heavily to manage data and interactions in PG listing, filters, and chat.

How is state management handled in your app?

We used **local state** via `useState` in most components.

For example:

- PG form data
- Search filter values
- Chat messages

We didn't need Redux or Context for this project, as the state was not shared across many levels.

Did you use Redux or Context API?

No. We used only **local state** (`useState`) and **props**.

Redux/Context would be useful in a much larger app with global shared state — ours was small and simple enough without it.

How did you implement responsive design?

We used **Tailwind CSS's responsive utility classes** like `md:` , `lg:` to control layout for different screen sizes.

We tested it in browser DevTools to make sure it looked good on mobile, tablet, and desktop.

How did you validate your forms?

We used basic JavaScript logic with conditional rendering:

- Required field checks
- Length or format checks (like email or phone)
- Alert messages or error states shown when invalid

This prevents submitting empty or incorrect data.

How did you handle dynamic routes?

We used React Router's `useParams()` hook to get the PG `id` from the URL.

Then we used that ID to fetch the corresponding PG data from Firestore and display it on the details page.

What is the folder structure of your project?

Our folder structure is modular:

```
src/
├── components/     → Reusable UI parts (Navbar, Card)
├── pages/          → Route-based views (Home, Login, Details)
├── firebase/       → Firebase config and helpers
├── App.jsx         → Main app logic
├── main.jsx        → Entry point
```

This keeps the code organized and scalable.

What happens if a route is not found?

We used a **catch-all route** in React Router:

```
<Route path="*" element={<NotFound />} />
```

If someone enters a wrong URL, they're shown a "404 Not Found" or custom message page.

BACKEND / FIREBASE-RELATED QUESTIONS – Answers

How does Firestore store data?

Firestore stores data in a **NoSQL structure**:

- **Collections** → Like tables
- **Documents** → Like rows, each with key-value data
- Inside documents, you can also have **subcollections**

Example: `pgListings` collection → each listing is a document with fields like title, rent, ownerId, imageURL, etc.

What are collections and documents in Firestore?

- A **collection** is a group of related documents (e.g., `pgListings`, `users`)
- A **document** is a single unit of data (like one PG listing)

- Documents have unique IDs and can hold fields, arrays, and nested data

It's a flexible and scalable format.

What are Firestore security rules?

Security rules control **who can read or write** what data in the database.
They run **before** the request is accepted.
Examples:

```
allow read: if request.auth != null;
allow write: if request.auth.uid == resource.data.ownerId;
```

This keeps data safe from unauthorized users.

How do Firebase rules protect your data?

They:

- Block unauthenticated access
- Restrict access based on roles (user vs owner)
- Allow users to only update their own data
  Even if someone modifies the frontend, rules on the backend will deny access if they aren't allowed.

How is user authentication handled?

We used **Firebase Authentication**, which supports:

- Email/Password login
- Google Sign-in
  When a user logs in, Firebase assigns them a **UID** that is used to track and secure their data.

How are user roles stored and used?

After signup, we store the user's role (`user` or `owner`) in the Firestore `users` collection.
When they log in, we fetch their role and:

- Show different UI features based on it

- Use the role in Firestore rules to control data access

How is data read/written to Firestore?

We use Firebase SDK functions like:

- `addDoc()` to write new documents
- `getDocs()` or `onSnapshot()` to read them
- `updateDoc()` to edit listings

  All operations are asynchronous and return promises.

What is the structure of your chat system in Firestore?

We use a `chats` collection:

- Each chat has a unique ID (e.g., userId + ownerId)
- Inside each chat document, there's a `messages` **subcollection**
- Messages are stored with sender ID, timestamp, and text

We listen in real-time using `onSnapshot()` .

How did you handle image uploads?

When an owner submits a PG, we:

1. Upload the image to **Firebase Storage**
2. Get the image's public download URL
3. Save that URL in Firestore with the listing data

We secure the storage using Firebase Storage rules.

What is Firebase Auth and how secure is it?

Firebase Auth is a managed authentication system.

- It securely stores hashed passwords
- It supports token-based login
- It prevents common attacks like brute-force or session hijacking

  It's widely used in production and trusted by companies.

What are the limitations of Firestore?

- Limited complex queries (can't filter on multiple fields unless indexed)
- Document size limit: 1MB
- Write limits: 1 write per document per second
- Free tier usage quotas (like read/write limits)

Still, it's suitable for most student or startup-level apps.

SECURITY & ACCESS CONTROL – Answers

How does your system ensure security?

We combine **frontend checks** with **Firestore security rules**:

- Only logged-in users can access features
- Only owners can post/edit their PGs
- Messages are only visible to sender and receiver
- Secure login using Firebase Auth
- Data access protected by rules and HTTPS

What is role-based access and how is it implemented?

We store each user's **role** (user or owner) in Firestore when they sign up.

- The **frontend** checks the role to show/hide features
- The **backend (Firestore rules)** uses that role to allow or deny access to collections like `pgListings`, `chats`

Can a user access owner-only features?

No.

- In the UI, user-role-based logic hides Submit PG and Dashboard for users
- In Firestore rules, users cannot write to the `pgListings` collection unless they're the owner of that document
  So even if they try using browser tools, access will be denied.

What prevents fake data submission?

We prevent it in three ways:

1. **Frontend validation** – only accept clean inputs
2. **User role check** – only owners can submit
3. **Firestore rules** – prevent users from writing unauthorized data

In future, we can also add admin approval or email verification.

How is your chat secured?

- Chats are stored under a `chats` collection
- Messages are in a `messages` subcollection with sender/receiver UID
- Firestore rules ensure that **only those two users** can read/write to the chat

No one else can access it.

How are passwords stored in Firebase?

Firebase Auth securely stores passwords using **hashing** and **salting**.
We never see or store passwords ourselves — it's all handled by Firebase.

What is HTTPS and why is it important?

HTTPS encrypts all data sent between browser and server using SSL/TLS.
This protects sensitive information like login credentials and messages from being intercepted or modified.
Both Vercel and Firebase use HTTPS by default.

Can someone hack your frontend and gain access?

They can see the code, but **can't bypass Firestore rules**.
Even if they edit the UI to try and submit data, Firebase will block unauthorized actions based on UID and rules.

What if someone tries to write data directly to Firestore?

If they're not authorized (by UID or role), **Firestore rules will block the request**.
Rules run on the server, not in the frontend — they're not bypassable by browser tricks.

How do you secure sensitive data during transmission?

All communication between frontend and Firebase is encrypted over **HTTPS**, and authenticated via secure tokens.
We also avoid exposing personal data unnecessarily and keep Firestore rules strict.

FEATURES / FUNCTIONALITY – Answers

What are the main features of MessFinder?

- Search and filter PG listings
- Post a room with image and details (for owners)
- Real-time chat between users and owners
- Role-based dashboards (user vs owner)
- Secure login and data access
- Mobile-responsive UI

How does the chat system work?

- When a user wants to talk to an owner, they click "Chat"
- A chat document is created (or reused) in Firestore
- Messages are stored in a subcollection
- Both sender and receiver see updates in real time using `onSnapshot()`
- Only those two users can access the chat

How does search and filter work?

- Users can enter a location or keyword
- We fetch data from Firestore and apply filters like:
    - Gender preference
    - Mess type
    - Price range

- We either use Firestore compound queries or filter on the client side

---

How does a user find a room?

- The user goes to the homepage or search page
- Types a keyword or selects filters
- The matching PGs are shown in a card layout
- They can click to view details or start a chat with the owner

---

How does a PG owner post a listing?

- Owner logs in and goes to "Submit PG"
- Fills in form fields like title, rent, type, facilities, and uploads an image
- On submit:
    - The image is uploaded to Firebase Storage
    - Listing data is saved in Firestore with the image URL

Can a user save PGs?

Not currently, but it can be added.
We can let users **bookmark** PGs by saving listing IDs in a subcollection under their user document.

How are images handled in the app?

- Owners upload PG images using a file input
- The image is stored in **Firebase Storage**
- We get a **secure image URL**
- That URL is saved in Firestore and displayed using `<img>` tags in the UI

Is real-time sync available?

Yes, in two main areas:

- **Chat**: Messages are synced in real time using Firestore `onSnapshot()`
- **Listings**: We can also show live updates to listings (optional)

How are listings verified?

Currently, listings are added by owners.
In the future, we can:

- Add admin review before publish
- Use email or phone number verification
- Allow users to report fake or outdated listings

Is there a review or rating system?

Not yet.
In future, we plan to:

- Let users leave one review per booking
- Store reviews in Firestore under each listing
- Show average ratings and comments on listing pages

DESIGN & DEVELOPMENT – Answers

How did you plan the system flow?

We first identified the **two roles**: user and owner.
Then we mapped out the core flows:

- For users: Search → View PG → Contact owner
- For owners: Login → Submit PG → Manage listings → Chat
  We created wireframes and then converted them into components and pages.

What is your development process?

We followed a simple Agile-like process:

1. Plan features and divide work
2. Build basic layout and routes
3. Add backend (Firebase) connections
4. Implement core features like chat, submit, search
5. Test and refine

6. Deploy and prepare documentation

Which SDLC model did you follow?

We used a **modified Waterfall + Iterative** model:

- Planned everything early (Waterfall style)
- Then built and tested features in cycles (Iterative)
  This worked well for our 2-month project with clear deliverables.

What were the stages in building your app?

1. **Requirement gathering**
2. **Designing UI and system flow**
3. **Firebase setup (Auth, Firestore, Storage)**
4. **Frontend development using React**
5. **Integrating all features**
6. **Testing on multiple devices**
7. **Final hosting and deployment**
8. **Documentation and presentation prep**

How did you test the application?

We tested:

- User flows (signup, search, chat)
- Form validations
- Role restrictions
- Image uploads
- Data fetching
- Responsiveness on different devices
  We used manual testing and DevTools for checking network and console.

What tools did you use for design?

We used:

- **Tailwind CSS** for UI design
- **Google Fonts & Icons**

- **Unsplash/SVGRepo** for images and icons
- (Optional: Figma/pen-paper sketches for flow planning)

What is the database schema?

It's a NoSQL schema:

- `users` → user info + roles
- `pgListings` → PG details with imageURL and ownerId
- `chats` → chat ID → `messages` subcollection
- Storage → image files named by timestamp or UID

How did you manage responsiveness?

Using:

- **Tailwind CSS responsive classes** (`sm:` , `md:` , `lg:`)
- **Flex and Grid layouts**
- **Viewport-based font and image sizing**
- We tested layouts on desktop, mobile, and tablet viewports

What are the different modules of your app?

1. **Authentication** (login/signup)
2. **Search and filter**
3. **PG submission and listing**
4. **Chat system**
5. **Dashboard (user & owner)**
6. **Role-based access control**
7. **Image handling and storage**

How are user flows managed?

We mapped out flows for both users:

- **User**: Login → Search → View PG → Chat
- **Owner**: Login → Submit PG → View listing → Chat
  React Router and conditional rendering manage these flows on the frontend.

DEPLOYMENT / HOSTING – Answers

Where did you host your frontend?

We hosted the frontend on **Vercel**, a modern platform for deploying React and frontend projects. Vercel connects directly to GitHub and handles builds and deployment automatically when code is pushed.

How does Vercel handle deployment?

Vercel uses a CI/CD pipeline:

- It watches our GitHub repository
- Every time we push new code, it builds the project
- Then it automatically deploys the new version to a secure URL
  No manual steps are needed after setup.

Where is your database hosted?

Our database (Firestore), authentication system, and file storage are all hosted by **Firebase**, which is Google's cloud backend platform.
It manages all data securely in the cloud without any server setup from our side.

Is your hosting free or paid?

Both **Vercel** and **Firebase** offer generous **free tiers**:

- Vercel: Free for hobby projects with unlimited deployments
- Firebase: Free up to 50K reads/day, 20K writes/day, 1GB Storage

We stayed within the free limits for our student project.

How do you update your app once deployed?

Whenever we change something in our code:

- We commit and push to GitHub
- Vercel automatically detects the change and redeploys the app

- The updated version goes live in under a minute

No need for FTP or manual uploads.

What is CI/CD and do you use it?

CI/CD stands for:

- **Continuous Integration** – Merging code frequently and testing
- **Continuous Deployment** – Automatically releasing code to users

Yes, we use CI/CD through Vercel and GitHub — push code → auto deploy.

What happens if Firebase or Vercel goes down?

If:

- **Firebase** goes down → Login, chat, or database actions will stop
- **Vercel** goes down → Frontend becomes inaccessible

In production, we'd implement **offline fallbacks**, **backups**, or **failover systems**.
For now, we rely on their high uptime and backups.

FUTURE PLANS / EXPANSION – Answers

What features do you plan to add in the future?

We plan to add:

- Map-based PG search
- Online payment for booking
- Review and rating system
- Notifications (push/email)
- Bookmark or save listing feature
- Admin panel for moderation

How will you implement map-based search?

We'll use the **Google Maps API** to:

- Show each PG on a map using its location
- Let users search PGs by area visually
- Allow location filters like "near college" or "within 1 km"

PG owners will enter a location when posting, which will be geocoded.

Will you add payment integration?

Yes. In the future, we can use:

- **Razorpay** or **Stripe** for secure payments
- Allow users to pay booking fees directly through the app
- Integrate it with owner dashboards and payment records

How can you implement review and rating?

We'll:

- Let users submit one review per PG they contact
- Store reviews in a subcollection under each PG document
- Display average ratings and user feedback
- Add moderation to remove abusive content

Can you build a mobile app version?

Yes, we can use:

- **React Native** to reuse most of our code
- Or Flutter (but it would need rewriting)

Firebase backend will remain the same — only frontend UI changes.

How will you moderate fake listings?

We plan to:

- Add an **admin dashboard**
- Let users report suspicious PGs
- Add phone/email verification for owners

- Possibly require address proof (optional future idea)

Will you add admin or moderation panel?

Yes. The admin will:

- View all listings
- Approve/reject listings before they go public
- Manage reviews or reported PGs
- Track user activity

Admins will be identified by role in Firestore.

How will you make the app more accessible?

We'll improve:

- Screen-reader support using ARIA labels
- Keyboard navigation
- Use semantic HTML for better structure
- Improve color contrast and font readability
- Allow users to change font size or switch to dark mode

How can you scale this to other cities?

It's simple — PGs are already location-based.

- We'll allow PG owners from any city to post
- Add filtering by city or PIN code
- Expand our database gradually
- Advertise through colleges or student groups

No major code change is required — just more data and broader promotion.

VIVA-WORTHY CONCEPTUAL TOPICS – Answers

Client vs Server in your architecture

- **Client**: The user's browser running React (your frontend)
- **Server**: Firebase (Firestore, Auth, Storage) that processes data, handles login, and stores PG/chat info

Client sends requests → Server responds with data or saves it.

REST API vs Firebase SDK

- **REST API**: Uses HTTP methods like GET/POST; you send requests manually.
- **Firebase SDK**: Provides ready-to-use JavaScript functions (like `addDoc`, `getDocs`) — no need to manually write REST calls.

Your project uses Firebase SDK.

Server-side rendering vs client-side rendering

- **Client-side rendering (CSR)**: Page loads once, then JS handles updates — used in SPAs like MessFinder.
- **Server-side rendering (SSR)**: Server sends fully built HTML for each request — faster first load, but harder to manage.

You use **CSR with React**.

NoSQL vs SQL

- **SQL (Structured)**: Uses tables, rows, and strict schemas (e.g., MySQL, PostgreSQL)
- **NoSQL**: Uses collections and documents with flexible structure (e.g., Firestore)

Firestore is NoSQL — great for flexible, nested, real-time data.

Hashing vs Encryption

- **Hashing**: One-way conversion (can't be reversed); used for passwords
- **Encryption**: Two-way conversion using a key; used to protect data during transfer

Firebase uses both: hashing for passwords and encryption for communication.

Firestore vs Realtime Database

- **Firestore**: More powerful, scalable, and structured (used in your app)
- **Realtime DB**: Older, flatter structure, good for very basic apps

Firestore allows queries, subcollections, better performance.

Difference between authentication and authorization

- **Authentication**: Verifies who you are (e.g., login)
- **Authorization**: Determines what you can access (e.g., can owner submit a PG?)

Your app uses Firebase Auth for authentication and Firestore rules for authorization.

Difference between localStorage and sessionStorage

- **localStorage**: Data stays even after closing browser
- **sessionStorage**: Data is cleared when tab/browser is closed

Firebase uses its own secure session tokens — not these storages directly.

State vs props in React

- **State**: Local, changeable data inside a component (`useState`)
- **Props**: Read-only data passed to a component from its parent

You use state for forms, filters; props to pass room data to components.

Public vs private routes

- **Public route**: Can be accessed by anyone (e.g., Home, Search)
- **Private route**: Requires login (e.g., Dashboard, Submit PG)

You implemented private routes using login checks and conditional rendering.

Static hosting vs dynamic hosting

- **Static hosting**: Files don't change (HTML, JS, CSS) — Vercel or Firebase Hosting
- **Dynamic hosting**: Server builds and sends content on the fly (like with Node.js or PHP)

Your app uses static hosting with dynamic data from Firebase.

CDN and how Vercel uses it

CDN (Content Delivery Network) stores app files across global servers.
When a user opens your app, the nearest CDN location serves it quickly.
Vercel uses CDN by default — improves speed and reduces load time.

PERSONAL CONTRIBUTION QUESTIONS – Answers

What was your personal contribution to the project?

(Example — customize as needed)
I worked mainly on the backend integration and core features like PG submission, Firebase data structure, chat functionality, and security rules. I also helped in deploying the app and testing it thoroughly.

Which feature or module did you build?

(Example — change based on your role)
I built the "Submit PG" form, image upload system using Firebase Storage, and the chat system using Firestore subcollections. I also worked on login flow and role-based routing.

What part was most challenging for you?

Understanding and writing proper **Firestore security rules** was tricky. It took time to figure out how to safely allow/deny actions based on roles, ownership, and login status.

How did you debug issues?

- I used **Chrome DevTools** and console logs to inspect errors
- Checked Firebase Console to monitor failed requests or rule violations
- Used test accounts to simulate different user actions
- Collaborated with teammates to trace bugs across pages

Did you work on both frontend and backend?

Yes. Our stack is full JavaScript, so we shared responsibilities.
I worked on both React components (forms, routes) and backend logic (Firestore data, rules, chat, auth).

How did you collaborate with your team?

We divided responsibilities clearly and communicated regularly.
We used GitHub for version control, and discussed bugs, design, and testing together. Everyone shared ideas and helped review each other's code or content.
